PyTorch x Scalar x Meta x OpenEnv Hackathon

1) What you need to learn first

From your CV, you already have a good base: **Python, Java, DS, SQL, Git/GitHub, full stack, ML basics, and UI/UX**. For this hackathon, the missing pieces are mainly these:

Must-learn

1. **OpenEnv basics**
   - what an environment is
   - `reset()`
   - `step(action)`
   - observation/state
   - reward
   - done/termination
   - how graders score outputs
2. **Docker basics**
   - writing a `Dockerfile`

- building an image
- running a container
- exposing a port
- debugging build errors

3. **Hugging Face Spaces**
   - creating a Space
   - pushing code
   - selecting Docker Space
   - checking logs
   - making sure the app is reachable
4. **Basic agent / RL thinking**
   - state → action → reward
   - episode
   - partial reward
   - baseline agent
   - deterministic scoring
5. **Good environment design**
   - clean task definition
   - stable reward
   - reproducible grader
   - edge-case handling
   - documentation

The dashboard explicitly requires a **real-world task**, **3 tasks with increasing difficulty**, a **meaningful reward function**, a **baseline inference script**, **Hugging Face Spaces deployment**, and a **README** with the environment description, actions, observations, rewards, running instructions, testing instructions, and baseline score details.

---

2) Rank of your 5 selected problem statements

I am ranking them for **Round 2 qualification chance** based on the dashboard's judging style: utility, novelty, ease of grading, and deployment stability.

Rank 1 — Motorsport Strategy Environment

Why: Most unique, easy for you to understand, strong novelty, and easy to make a clear reward structure.

Risk: Needs a good simulation design, but that is manageable.

Rank 2 — ICU Alert Prioritization

Why: Very strong real-world utility, high impact, serious domain value.

Risk: More complex because medical logic must be carefully designed.

Rank 3 — Data Cleaning Decision Environment

Why: Very practical, easy to explain, easy to grade, very solid for judges.

Risk: Less flashy than motorsports, so you need good novelty in design.

Rank 4 — Notification Overload Manager

Why: Huge userbase, very relatable, easy to simulate.

Risk: Can look too generic unless you make the environment clever.

Rank 5 — Deadline Risk Predictor

Why: Useful for students and teams, simple to understand.

Risk: Harder to make feel like a deep environment unless you add good action/reward design.

Final recommendation

Choose Motorsport Strategy if your goal is the strongest solo submission. It gives you the best combination of:

- uniqueness

- personal interest
- clean reward design
- easier storytelling
- better novelty score

---

3) Full execution plan

Step 1: Learn the tools you need

OpenEnv

Learn:

- environment structure
- action space
- observation space
- reward design
- `reset()` and `step()`
- how graders test outputs

How to learn:

- read the hackathon prep modules
- build one tiny toy environment locally first
- practice turning text input into action and reward
- test with a simple rule-based baseline

Docker

Learn:

- `Dockerfile`
- `WORKDIR`
- `COPY`
- `RUN pip install ...`

- `EXPOSE`
- `CMD`

How to learn:

- create a simple Python app
- dockerize it
- run it locally
- fix errors until it starts cleanly

Hugging Face Spaces

Learn:

- create a Space
- use Docker Space
- upload code
- read logs
- verify the environment is live

How to learn:

- deploy a very small test app first
- then move to your environment
- keep the app lightweight and stable

Basic agent / RL thinking

Learn:

- an agent sees a state
- takes an action
- gets a reward
- task ends or continues
- the goal is to maximize total reward

How to learn:

- think in tiny loops: `state -> action -> reward -> next_state`

- compare good vs bad actions
- make rewards partial, not all-or-nothing

Step 2: Pick one PS and lock it

For you, lock this:

Selected PS: Motorsport Strategy Environment

Why this is the best solo choice

- You already like motorsports
- judges see it as unique
- it can be made into a real-world planning environment
- reward can be made clean and measurable
- it can be expanded into 3 tasks easily

---

4) Define 3 tasks for each PS

Below, each PS has:

- **Easy**
- **Medium**
- **Hard**

Each one can be turned into an OpenEnv-style environment.

A) ICU Alert Prioritization

Idea

Doctors get too many alerts. The environment makes an AI decide which alerts to escalate first.

Actions

- `0 = ignore`
- `1 = monitor`
- `2 = escalate`
- `3 = urgent escalation`

Task 1 — Easy

Single alert, obvious severity.

- Example: oxygen drop, heart rate spike, normal warning
- Goal: classify correctly

Task 2 — Medium

Multiple alerts at once.

- Goal: prioritize top 2 alerts correctly

Task 3 — Hard

Noisy alerts with missing context.

- Goal: choose the most critical alert with limited information

Reward design

- correct urgency = +1
- wrong urgency = -1
- partial correct priority = +0.5
- missing the critical alert = strong penalty

B) Notification Overload Manager

Idea

AI decides whether a notification should be shown now, delayed, bundled, or suppressed.

Actions

- `0 = show now`

- `1 = delay`
- `2 = suppress`
- `3 = bundle`

Task 1 — Easy

Single notification, obvious importance.

- Example: OTP, promo, news, social ping

Task 2 — Medium

Multiple notifications with user context.

- Goal: reduce interruption while keeping important alerts visible

Task 3 — Hard

Time-sensitive stream with competing priorities.

- Goal: balance usefulness vs distraction

Reward design

- important notification shown at right time = +1
- irrelevant notification suppressed = +1
- wrong delay/suppression = penalty
- too many interruptions = negative score

C) Motorsport Strategy Environment

Idea

AI acts like a race strategist.

Actions

- `0 = pit`
- `1 = stay out`
- `2 = change tyre strategy`
- `3 = conserve tyres`

- `4 = push hard`

Task 1 — Easy

Simple lap-by-lap pit decision.

- Goal: pit at the correct time

Task 2 — Medium

Weather and tyre wear included.

- Goal: decide between staying out and pitting based on conditions

Task 3 — Hard

Race has safety car / tyre degradation / traffic.

- Goal: optimize total race time

Reward design

- optimal lap time improvement = positive reward
- bad pit timing = penalty
- tyre wear management = partial reward
- race finish time better than baseline = high score

D) Data Cleaning Decision Environment

Idea

AI decides how to clean a messy dataset.

Actions

- `0 = keep`
- `1 = drop row`
- `2 = fill missing`
- `3 = transform`
- `4 = flag anomaly`

Task 1 — Easy

Simple missing values.

- Goal: identify obvious cleanup action

Task 2 — Medium

Mixed missing values + outliers.

- Goal: choose the best transformation

Task 3 — Hard

Messy multi-feature dataset with noise.

- Goal: improve final data quality score

Reward design

- correct cleanup = +1
- destructive cleanup = penalty
- better final dataset quality = higher reward

E) Deadline Risk Predictor

Idea

AI predicts whether a task will miss deadline and suggests a fix.

Actions

- `0 = on track`
- `1 = at risk`
- `2 = delayed`
- `3 = suggest intervention`

Task 1 — Easy

One task, obvious time risk.

Task 2 — Medium

Multiple tasks with deadlines and workload.

Task 3 — Hard

Changing priorities, missing info, and partial progress.

Reward design

- accurate risk prediction = +1
- useful intervention = +1
- wrong prediction = penalty
- better schedule outcome = extra reward

---

5) How to define the environment properly

This is the most important part.

Environment = a structured simulation

You need these pieces:

1. State / Observation

What the agent sees.

Example for Motorsport:

- current lap
- tyre wear
- fuel level
- weather
- safety car status
- gap to competitor

2. Actions

What the agent can do.

Example:

- pit

- stay out
- switch tyre compound
- conserve tyres
- push hard

3. Reward

What the agent gets after action.

Reward must:

- be meaningful
- not be random
- reward progress
- punish bad decisions
- have partial scoring

4. Reset

Starts a fresh episode.

Example:

- reset race at lap 1
- reset tyre wear
- reset weather/track state
- reset score

5. Step

Takes one action and returns:

- next state
- reward
- done
- info

6. Done

Episode ends when:

- race is finished
- task is solved
- maximum steps reached

Simple environment skeleton

```
classMyEnv:
def__init__(self):
self.max_steps=10
self.current_step=0
self.state=None

defreset(self):
self.current_step=0
self.state=self._initial_state()
returnself.state

defstep(self,action):
reward=self._compute_reward(action,self.state)
self.state=self._transition(self.state,action)
self.current_step+=1
done=self.current_step>=self.max_steps
info= {}
returnself.state,reward,done,info
```

How to do it correctly

- keep the state simple enough to test
- keep actions limited and meaningful
- make rewards deterministic
- add difficulty levels gradually
- make the environment reproducible
- make sure `reset()` always gives a clean start
- make sure `step()` never breaks

The dashboard specifically expects a **full OpenEnv spec**, a **meaningful reward function**, and **reproducible baseline scores**.

---

6) Baseline inference script sample

The dashboard says the baseline script should run reproducibly, use environment variables such as `API_BASE_URL`, `MODEL_NAME`, and `HF_TOKEN`, and be named `inference.py` in the project root. It also says the script should work with OpenAI-style client calls.

Here is a simple sample baseline script:

```
importos
importre
fromopenaiimportOpenAI

API_BASE_URL=os.getenv("API_BASE_URL")
MODEL_NAME=os.getenv("MODEL_NAME")
HF_TOKEN=os.getenv("HF_TOKEN")

client=OpenAI(
api_key=HF_TOKEN,
base_url=API_BASE_URL
)

ACTION_MAP= {
"motorsport": {
0:"pit",
1:"stay out",
2:"change tyres",
3:"conserve tyres",
4:"push hard"
    }
}

defparse_action(text):
match=re.search(r"\b([0-4])\b",text)
ifmatch:
returnint(match.group(1))
return1# safe default
```

```
defchoose_action(observation,task_name="motorsport"):
prompt=f"""
You are an agent in a real-world environment.
Observation:
{observation}

Choose one action only.
0 = pit
1 = stay out
2 = change tyres
3 = conserve tyres
4 = push hard

Return only the number.
"""

response=client.chat.completions.create(
model=MODEL_NAME,
messages=[
            {"role":"system","content":"You are a careful d
ecision-making agent."},
            {"role":"user","content":prompt}
        ],
temperature=0
    )

text=response.choices[0].message.content.strip()
returnparse_action(text)

if__name__=="__main__":
sample_observation="Lap 18, tyre wear high, rain expected i
n 5 laps, gap to car behind is 1.2s"
action=choose_action(sample_observation)
print("Chosen action:",action)
```

How to use this

- replace the prompt with your actual task text
- keep temperature at `0`
- parse actions safely
- always return a valid action
- make it deterministic

This is only a baseline. Your final project should have a stronger decision policy or task-specific heuristics.

---

7) Deploy to Hugging Face Spaces

The dashboard requires deployment to **Hugging Face Spaces** and says the environment must run in a containerized Space with a working Dockerfile.

Deployment steps

Step 1

Create a Hugging Face Space.

Step 2

Choose **Docker** as the Space type.

Step 3

Add these core files:

- `app.py` or main entry point
- `inference.py`
- `Dockerfile`
- `requirements.txt`
- `README.md`

Step 4

Push the code to the Space repository.

Step 5

Open logs and fix errors until the Space launches.

Step 6

Verify the Space URL returns correctly.

Step 7

Run the validator locally before submitting.

Dockerfile shape

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE7860

CMD ["python","app.py"]
```

Deployment tips

- keep dependencies minimal
- avoid huge models unless necessary
- make the app start fast
- ensure all env vars are documented
- keep the Space stable on restart

---

8) README format

The dashboard says your README must include environment description, action and observation definitions, task descriptions, setup instructions, and baseline

scores.

Use this exact format:

```
# Project Title

## 1. What the environment does
Explain the real-world problem in 3–5 lines.

## 2. Why this problem matters
Explain the user base and real-world use.

## 3. Environment overview
Explain the simulation loop:
state -> action -> reward -> next state

## 4. Observation space
List all inputs the agent sees.

## 5. Action space
List all possible actions and what each means.

## 6. Reward function
Explain how rewards are computed.
Mention:
- correct action reward
- wrong action penalty
- partial reward
- success criteria

## 7. Tasks
### Task 1: Easy
### Task 2: Medium
### Task 3: Hard

For each task, write:
- goal
- observation example
- action example
```

```
- reward idea

## 8. How to run
Explain local run steps:
- install dependencies
- start app
- test environment
- run baseline inference

## 9. How to test
Explain how to run validation checks:
- Docker build
- local environment test
- baseline script test
- Space URL test

## 10. Baseline score
Show baseline results on each task.
Mention reproducibility.

## 11. File structure
List important files.

## 12. Notes
Explain any special setup or assumptions.
```

Keep the README simple, direct, and clean.

---

9) Validate before submission

The dashboard is strict here. It says the Space must deploy, OpenEnv compliance must pass, the Docker build must work, the baseline inference script must run, and submissions that do not deploy or are trivially modified/plagiarized can be disqualified. It also says only the latest submission before the deadline is evaluated.

Validation checklist

A. Local code check

- run your environment locally
- call `reset()`
- call `step()` a few times
- ensure no crashes

B. Docker check

- build the container
- run the container
- confirm the app starts

Commands:

```
docker build-t hackathon-env .
docker run-p7860:7860 hackathon-env
```

C. Baseline check

- run `inference.py`
- confirm it prints a valid action
- confirm it does not crash
- confirm it is deterministic

D. Environment spec check

- verify `reset()` returns a valid initial observation
- verify `step(action)` returns:
  - next state
  - reward
  - done
  - info

E. Task check

- confirm 3 tasks exist
- confirm difficulty increases

- confirm each task has a meaningful score

F. Reward check

- confirm reward is not trivial
- confirm partial progress is rewarded
- confirm bad actions are penalized

G. Space deployment check

- open the Space URL
- confirm it loads
- confirm logs show no startup error
- confirm the app responds

H. Final submission check

- submit only when everything passes
- submit the latest version
- do one final end-to-end run before final upload

The dashboard also lists a pre-submission checklist with HF Space deployment, OpenEnv compliance, baseline reproducibility, 3 tasks with graders, and a runtime limit for validation.

---

10) What “winning” means here

Use this formula:

```
Real-world task
↓
Convert into environment
↓
Define state + action + reward
↓
Create 3 tasks
↓
Build grader
```

```
↓
Deploy
```

And then optimize for these 5 things:

- unique idea
- clear reward function
- 3 difficulty levels
- clean grader
- smooth deployment

That is the strongest path to Round 2.

---

11) Rank of your 5 PS in one line each

1. **Motorsport Strategy** — best overall for you
2. **ICU Alert Prioritization** — strongest utility
3. **Data Cleaning Decision** — most practical
4. **Notification Overload Manager** — biggest userbase
5. **Deadline Risk Predictor** — simplest student-use case

---

12) TXT snippet for your doc

Copy this into a `.txt` or `.md` file:

```
Hackathon execution flow:

1. Learn OpenEnv, Docker, Hugging Face Spaces, and basic ag
ent/RL thinking.
2. Pick one real-world problem statement.
3. Define the environment: state, action, reward, reset, st
ep, done.
4. Create 3 tasks with increasing difficulty.
5. Build a deterministic baseline inference script.
6. Deploy the environment to Hugging Face Spaces using Dock
```

```
er.
7. Write a strong README with setup, tasks, actions, observ
ations, rewards, and baseline scores.
8. Validate everything before submission.
9. Submit the latest working version only.

Winning formula:
Unique idea + Clear reward function + 3 difficulty tasks +
Clean grader + Smooth deployment = Round 2 qualification
```

---

13) Super master mega prompt for Antigravity

Use this as a full-generation prompt for your project tool:

```
You are an expert hackathon project architect, AI environme
nt designer, full-stack Python engineer, Docker specialist,
Hugging Face Spaces deployer, and RL-style evaluation syste
m builder.

I want you to generate a complete, submission-ready hackath
on project for the Scaler x Meta x PyTorch x Hugging Face O
penEnv hackathon. The project must be fully designed for a
solo participant and should maximize the chance of qualifyi
ng for Round 2. The solution must follow the hackathon rule
s shown in the dashboard:
- Build a complete, real-world OpenEnv environment that an
AI agent can learn from through the standard step(), reset
(), and state() API.
- The environment must simulate a real-world task, not a to
y game.
- Implement full OpenEnv spec: typed models, step/reset/sta
te, OpenEnv-compatible structure.
- Include at least 3 tasks with agent graders; difficulty m
ust increase from easy to medium to hard.
- Include a meaningful reward function with partial progres
s signals.
```

```
- Include a baseline inference script using environment var
iables and an OpenAI-compatible client.
- Deploy the environment to Hugging Face Spaces using Docke
r.
- Write a README with environment description, action/obser
vation definitions, setup instructions, usage, baseline sco
res, and testing steps.
- Ensure the project is reproducible, deterministic, and ea
sy to validate.

Project theme:
Build a Motorsport Strategy Environment.
This should simulate racing strategy decisions such as pit/
no pit, tyre choice, tyre conservation, and push-hard decis
ions.
The environment must be realistic enough to feel like a rea
l-world decision system and should stand out from typical h
ackathon ideas.

Project requirements:
1. Create a clear problem statement.
2. Define why the problem matters in real life.
3. Define the user base.
4. Define the environment state.
5. Define the action space.
6. Define the reward system.
7. Define episode termination rules.
8. Define 3 tasks:
   - Task 1: Easy pit stop decision
   - Task 2: Medium weather and tyre wear strategy
   - Task 3: Hard race strategy with safety car / traffic /
tyre degradation
9. For each task, define:
   - initial state
   - available actions
   - expected behavior
   - reward logic
   - success criteria
```

```
10. Add a baseline agent:
   - rule-based or prompt-based
   - must be deterministic
   - must be simple, reproducible, and runnable
11. Add grader functions:
   - deterministic scoring
   - no randomness in evaluation
   - consistent outputs
12. Add local test scripts:
   - verify reset()
   - verify step()
   - verify baseline inference
   - verify Docker startup
13. Add Hugging Face Spaces deployment files:
   - Dockerfile
   - requirements.txt
   - app entry point
   - inference.py
   - README.md
14. Add clear file structure.
15. Add comments in code so a beginner can understand it.

Design constraints:
- Keep the project realistic but simple enough to finish so
lo.
- Prefer clean, explainable logic over complex ML.
- Avoid unnecessary dependencies.
- Make the code readable and modular.
- Use typed Python code where possible.
- Make sure the environment can be validated automatically.
- Make sure the Space starts reliably.
- Make sure the project can be explained in a demo or viva
in simple language.

Scoring strategy:
Optimize for:
- real-world utility
- task and grader quality
```

```
- environment design
- code/spec compliance
- creativity and novelty

Expected output from you:
A complete project blueprint containing:
- problem statement
- architecture
- environment design
- task definitions
- reward design
- grader logic
- baseline inference logic
- repo folder structure
- Dockerfile
- README template
- deployment instructions
- validation checklist
- final submission checklist

Also produce:
1. A concise explanation for each file in the repo.
2. A starter code skeleton for the environment.
3. A starter code skeleton for the baseline inference scrip
t.
4. A starter code skeleton for the grader.
5. A starter Dockerfile.
6. A starter README.
7. A final submission plan for solo execution in 3 phases:
   - phase 1: learning and design
   - phase 2: implementation and testing
   - phase 3: deployment and submission

Important:
The explanation must be simple, beginner-friendly, and dire
ctly usable by one person working alone.
The final answer should be structured, actionable, and comp
```

```
lete enough that the project can be built immediately after
reading it.
```

---

14) Final conclusion for you

The best solo path is:

Learn the tools → pick Motorsport Strategy → build 3 difficulty levels → make a deterministic reward/grader → deploy on HF Spaces → validate hard → submit only after full end-to-end testing.